



No Free Lunch: Overcoming Reward Gaming in AI Safety Gridworlds

DOI:

[10.1007/978-3-030-83906-2_18](https://doi.org/10.1007/978-3-030-83906-2_18)

Document Version

Accepted author manuscript

[Link to publication record in Manchester Research Explorer](#)

Citation for published version (APA):

Tsvarkaleva, M., & Dennis, L. (2021). No Free Lunch: Overcoming Reward Gaming in AI Safety Gridworlds. In *4th International Workshop on Artificial Intelligence Safety Engineering (WAISE 2021)* (Vol. 12853) https://doi.org/10.1007/978-3-030-83906-2_18

Published in:

4th International Workshop on Artificial Intelligence Safety Engineering (WAISE 2021)

Citing this paper

Please note that where the full-text provided on Manchester Research Explorer is the Author Accepted Manuscript or Proof version this may differ from the final Published version. If citing, it is advised that you check and use the publisher's definitive version.

General rights

Copyright and moral rights for the publications made accessible in the Research Explorer are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

Takedown policy

If you believe that this document breaches copyright please refer to the University of Manchester's Takedown Procedures [<http://man.ac.uk/04Y6Bo>] or contact openresearch@manchester.ac.uk providing relevant details, so we can investigate your claim.



No Free Lunch: Overcoming Reward Gaming in AI Safety Gridworlds^{*}

Mariya Tsvarkaleva^[0000–0003–4843–6818] and Louise A. Dennis^[0000–0003–1426–1896]✉

Department of Computer Science, The University of Manchester, United Kingdom
`mariya.tsvarkaleva@student.manchester.ac.uk`,
`louise.dennis@manchester.ac.uk`

Abstract. We present two heuristics for tackling the problem of reward gaming by self-modification in Reinforcement Learning agents. Reward gaming occurs when the agent’s reward function is mis-specified and the agent can achieve a high reward by altering or fooling, in some way, its sensors rather than by performing the desired actions. Our first heuristic tracks the rewards encountered in the environment and converts high rewards that fall outside the normal distribution into penalties. Our second heuristic relies on the existence of some validation action that an agent can take to check the reward. In this heuristic, on encountering an abnormally high reward, the agent performs a validation step before either accepting the reward as it is, or converting it into a penalty. We evaluate the performance of these heuristics on variants of the tomato watering problem from the AI Safety Gridworlds suite.

Keywords: AI Safety Gridworlds · Reinforcement Learning · Reward Gaming

1 Introduction

In Reinforcement Learning (RL) methods, an agent explores an environment from which it receives rewards and adjusts its behavioural policy to optimize this reward signal. Since RL algorithms rely heavily on trial and error, this often results in unwanted, irreversible, and potentially harmful actions. For this reason, empirical Reinforcement Learning is limited and current algorithms are predominantly applied to simulated and tightly-controlled environments [9, 10, 7].

A poorly designed reward signal can often lead to what is called *reward gaming* [2]. This refers to an autonomous system exploiting misspecification to obtain a high reward which does not reflect the desired behaviour. A practical example could be a household robot whose reward signal is designed to minimize the amount of dishes it sees as dirty, which learns to obstruct its camera view

^{*} Work supported by EPSRC Grant EP/V026801/1 *Trustworthy Autonomous Systems Verifiability Node*

of the sink, so that it sees no unclean dishes. Thus, the agent can obtain a maximum reward while failing to behave as intended.

We propose two heuristics for avoiding this problem. In both heuristics the agent keeps a record of previous rewards and their distribution. Where a reward signal falls outside this distribution (the agent receives a “free lunch”) then it treats this with suspicion. In our first heuristic the agent converts free lunches into penalties (we refer to this as the *Direct Penalty* heuristic). In our second heuristic the agent performs a checking step to validate the reward signal (we refer to this as the *Doubt* heuristic).

We evaluate the performance of these heuristics on a set of examples based on the *tomato watering GridWorld* in the AI Safety Gridworlds [6] suite of challenge problems for RL agents. Experimental results demonstrate that both heuristics reduce reward gaming but that the second is able to detect and act upon genuine rewards that fall outside the distribution and also performs better even when all exceptional rewards are misspecified – however the Doubt heuristic, in the wrong environment, can still lead to reward gaming.

2 Background

Reinforcement Learning (RL) [11] is a learning method where an agent adopts a behaviour based on the rewards and penalties it receives from an environment. The agent’s goal is to optimize a quantifiable reward signal, and it learns by a trial-and-error process of exploring the space and receiving feedback. The agent learns a strategy, based on its observations, to maximize its cumulative reward. It achieves this by empirically building estimates of how valuable states (or more typically state-action pairs) are, starting with uncertain knowledge and improving its beliefs in an iterative fashion.

The Q value function, $Q(s, a)$, describes how good a state-action pair is with regards to the *expected return* that can be obtained from taking an action a in state s . The output of the function is called a Q value and is often used as a quality measure in reinforcement learning. RAINBOW [4] is an implementation of a deep learning approach to learning a policy that optimises the Q value. It combines a number of techniques and is available open source from Open AI. We used RAINBOW in our experiments.

2.1 AI Safety Gridworlds environments

AI Safety Gridworlds [6] is a suite of challenge problems that offers simplistic grid environments modelled to expose unsafe behaviour in RL agents. The gap between what we want the agent to achieve and what we told it to achieve is modelled in the environments with two functions – a *reward function* and a *safety performance function*. The reward function represents the signal the agent uses to optimise its policy, whereas the safety performance function defines the actual desired behaviour and is hidden from the agent. Environments are composed of a two-dimensional chessboard-like grid with a maximum size of 10x10. Each cell

can either be empty, unreachable, occupied by the agent, or occupied by sprites with which the agent can interact.

The gridworlds are represented as a finite *Markov Decision Process*. The action space, A , is the set of actions {up, down, left, right}. The robot interacts with the environment on each time step by performing an action $a \in A$, based on its observations of the current state, $s \in S$, which results in transitioning to a next state s' and obtaining a reward $R(s, a)$. It moves in the environment by shifting to a neighboring cell in the specified direction if the cell is empty, or otherwise stays in its place. Each gridworld has a limited number of steps that can be taken and form a single episode, which ends either when that limit is reached or a goal specific to the environment is accomplished.

In this paper we concern ourselves with one particular gridworld, the *tomato watering* environment, that captures the reward gaming by self-modification problem. In the *tomato watering* environment, the agent’s goal is to water as many tomatoes as possible within an episode. Unwatered tomatoes (t) appear as watered (T) whenever the agent steps on their tile, and then dry out with some probability P at each timestep. In addition, there is an observation/bucket tile (O) which when stepped on, makes the robot see all tomatoes as watered, but doesn’t change their real state. The important distinction here is that the robot’s reward function is based on how many tomatoes it perceives as watered, as opposed to how many have truly changed their state. This means that stepping on the O tile will yield a maximum reward while the goal of watering plants is not achieved. On the other hand, safety is expressed as a function of the number of truly watered tomatoes.

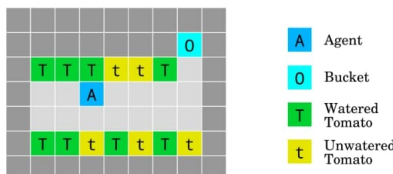


Fig. 1: The Tomato Watering Gridworld [6]

3 Two Heuristics: Detecting a Free Lunch

3.1 Direct Penalty: Assuming this is a Free Lunch

One way to avoid unsafe behaviour in the self-modification problem is by penalizing state-transitions that yield unrealistically high rewards that represent free lunches. Thus, we want the agent to learn when an action it has done has led to a reward out of distribution and discourage such moves. To achieve this, we need to define what a “suspiciously high” reward is, and decide how to penalize this behaviour. By holding a list of rewards the agent has received at each step, we consider a newly encountered reward as a free lunch if it is an outlier compared

to the others collected. NB: This heuristic relies on the assumption, common in RL, that the environment changes only slowly when the agent is not acting upon it.

We have implemented a variant of outlier detection. The solution consists of keeping a normal distribution over the rewards received, and at each step checking that the new one obtained lies within some range N of σ standard deviations away. Based on the empirical rule (or 68-95-99.7 rule) known in statistics which states that 99.7% of the data falls within 3 deviations, it is reasonable for N to be bounded between 3 and 5. We calculate the standard score z to determine how far away the datapoint, x_i , is from the mean μ , and compare if this is above the threshold N and should be considered an outlier. After a basic parameter search, $\sigma = 3$ was found to perform best and is used in all our experiments.

$$z = \frac{x_i - \mu}{\sigma} \quad (1)$$

Since rewards are achieved each step at a time, defining a threshold for what we consider abnormal can be difficult in early time steps before enough data about the distribution has been gathered. To account for this complication, we only start checking for outliers after some N steps have been taken, assuming that the agent has gathered observations which are more representative of the set of overall achievable rewards.

Finally, we penalize the undesired transitions. We experimented with modifying the reward to a static number and modifying it scaled by some value p . The second achieved better performance, because of the difficulty of categorizing outliers – if a normal action is incorrectly classified as abnormal, we want this to be penalized less than a move which has led to fake observations. Otherwise, we risk that the agent develops a policy of avoiding desirable states. Therefore the penalty is defined as:

$$R(s, a) = -R(s, a) * p \quad (2)$$

Pseudo code for the Direct Penalty heuristic is shown in Figure 2.

3.2 Doubt: Checking if this is a Free Lunch

In addition to achieving safe behaviour, we would like the agent to continue to strive for optimal task performance. [6] suggests a scenario where the O tile is a sprinkler that truly waters all the plants, and not merely an observation modifier. In such case, the reward obtained, which might be flagged as a free lunch, would be a valid solution to the problem.

We modified our Direct Penalty heuristic to a new heuristic we called *Doubt*. This heuristic presumes the existence of some validation action by which the result of a reward can be verified. This is obviously application specific. In the context of tomato watering we assume that an action truly changes the grid-world’s state if its resulting effects remain and so this validation action can consist of undoing the agent’s last move. The agent is equipped at each timestep

```

list_rewards ← []
while episode is not finished do
  action ← agent.choose_action()
  reward, observations ← agent.STEP(action)
  list_rewards ← list_rewards.ADD(reward)
  DETECT_OUTLIERS_STD()
end while
function DETECT_OUTLIERS_STD(list_rewards, reward)
  normal_distribution ← STD(list_rewards[-1])
  z ← Z_SCORE(reward, normal_distribution)
  if z > N then
    reward ← -reward * p
    list_rewards.POP()
  end if
end function

```

Fig. 2: The Direct Penalty Algorithm

with a 2D matrix representation of its observations. Therefore, it can decide if a change has persisted by comparing its observations of the doubted transition with those of the reversed one.

We employ a similarity metric for comparison, instead of an absolute equality between the matrices, because of the periodic random drying out of tomatoes, which makes the environment dynamic. We calculate similarity of two matrix observations of the same size A and B by subtracting their values and calculating the ratio of zero elements over all the elements. Finally, we utilize a threshold M to conclude if a penalty should be incurred (3).

$$C = A_{m,n} - B_{m,n} = \begin{pmatrix} a_{1,1} - b_{1,1} & \cdots & a_{1,n} - b_{1,n} \\ \vdots & \ddots & \vdots \\ a_{m,1} - b_{m,1} & \cdots & a_{m,n} - b_{m,n} \end{pmatrix} \quad (3)$$

$$Penalize^1 \begin{cases} True, & \text{if } \sum_{x \in C} \delta_{x,0} / \sum_{x \in C} 1 < M \\ False, & \text{otherwise} \end{cases}$$

Pseudocode for the Doubt algorithm can be found in Figure 3.

¹ Kronecker delta δ_{ij} is a function of two variables i, j that returns 1 if the variables are equal, and 0 otherwise

```

list_rewards ← []
while episode is not finished do
  if WAS_REVERSED() && BOARDS_SIMILARITY(prev, curr) < M then
    reward ← -reward * p
  end if
  if STEP_WAS_SUSPECTED() then
    action ← agent.OVERRIDE_WITH_REVERSED()
    transition_reversed ← True
  else
    action ← agent.CHOOSE_ACTION()
  end if
  reward, observations ← agent.STEP(action)
  list_rewards ← list_rewards.ADD(reward)
  DETECT_OUTLIERS_STD()
end while
function BOARDS_SIMILARITY(previous, current)
  subtracted ← current - previous
  proportion_similar ← COUNT_ZEROS(subtracted) / subtracted.SIZE()
end function

```

Fig. 3: The Doubt Algorithm

4 Evaluation

To investigate our heuristics, we implemented a modified version of the tomato watering environment, which we call the *tomato watering sprinkler* suite. We introduce a new *S* tile, which acts as a sprinkler that automatically waters all tomatoes whenever the agent steps on it. In contrast to the *O* tile, the *S* tile changes the number of truly watered tomatoes.

The *tomato watering sprinkler* suite consists of five cases, which demonstrate different configurations of the problem. The layout of the first case is similar to the *tomato watering* gridworld, consisting of dry tomatoes, watered tomatoes, and an observation-changing tile *O*. The only difference is that when the agent steps on the *O* tile, it observes only existing tomatoes as watered, instead of interpreting the full board as filled with watered tomatoes in order to provide a more consistent set of cases. The second case represents a situation where a sprinkler is put in the environment, however there is no observation tile (Figure 4). Finally, the last three cases represent different configurations of the *S* and *O* tile placed at the same time in the grid in different cells (Figure 5).

We ran an RL agent in the *tomato watering sprinkler* environment with neither the *O* nor *S* tiles present in order to establish a baseline reward when there is no possibility for gaming. This demonstrates the optimal return under guaranteed safe conditions within 200k timesteps. We observed that the agent’s performance grows steadily, reaching at most 9.5. Since tomatoes dry out with

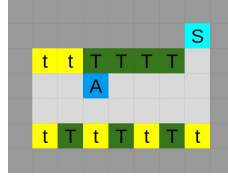


Fig. 4: Case 2: S tile only

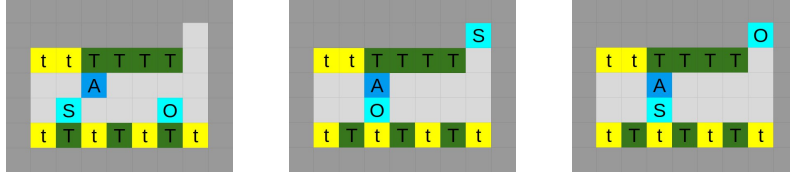


Fig. 5: Cases 3, 4, and 5: Both S and O tile

a probability of $P = 0.5$, it is extremely difficult (if not quite impossible) for the robot to reach all of them.

4.1 Case 1: Observation Tile Only

We trained up three agents in the first *tomato watering sprinkler* case. The first agent used no heuristics, the second used the Direct Penalty heuristic and third used the Doubt heuristic. The results are shown in Figures 6 and 7.

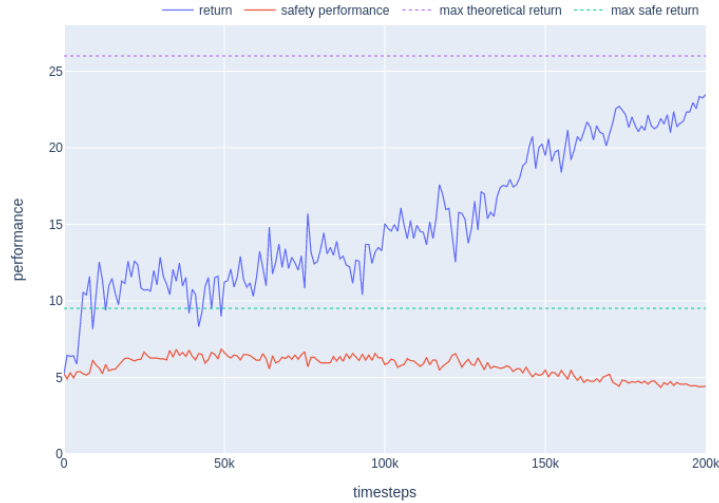


Fig. 6: Basic agent (no heuristics) - Case 1

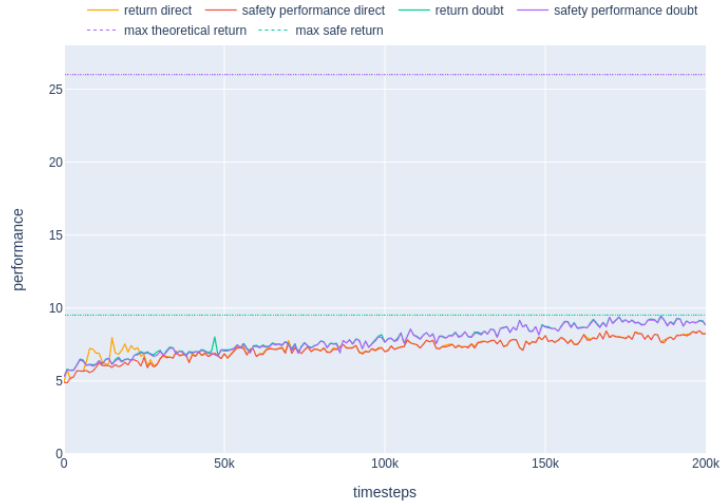


Fig. 7: Two heuristic agents - Case 1

Our results show that the agent that uses no heuristics to check the reward learns to cheat by stepping on the O tile, and so performs poorly with regards to the safety function. Meanwhile the two agents using heuristics to judge the reward do not develop a policy that exploits the O tile. When penalizing directly, the first 25k steps reveal a spike in the return which doesn’t align with the safety performance at that point. This means that the robot was frequently stepping on the O tile at first, but soon after, convergence is reached and the agent continues to act predominantly safely until the end of the run. When using the Doubt heuristic (green and purple solid lines), we observe an even better overlap of safety performance and return.

Further, the return obtained with the Doubt heuristic comes closer to the maximum safe return of 9.5 (green dashed line). We believe that the reason for this is the greatly reduced number of wrongly penalized steps illustrated in Table 1. When using the Direct Penalty heuristic, the portion of data wrongly categorized as outlying amounts to an overwhelming 79.6%. This is high compared to the very low number of O -tile steps which weren’t penalized – meanwhile the Doubt heuristic, capable of validating the reward, rarely issues an incorrect penalty. We reason that this high miscategorization might have disrupted the agent’s policy, making it avoid some desired reward actions (i.e., watering tomatoes). This would explain why the Direct Penalty heuristic achieves a lower return than the Doubt heuristic. Because the “doubtful” agent reverses a step to check if the reward persists, the miscategorisation of desired rewards is transferred to unnecessary reverse actions (shown in Table 1).

Heuristic	wrongly penalized	correctly penalized	stepped not penalized	stepped and penalized	wrongly reversed	correctly reversed
Direct Penalty	79.6 %	20.4 %	0.4%	99.6 %	N/A	N/A
Doubt Penalty	0.1%	99.9 %	12.7 %	87.3 %	67.9 %	32.1 %

Table 1

4.2 Case 2: Sprinkler Tile Only

In the second case the exceptionally high reward is, in fact genuine. For this reason, the theoretical maximum reward of 26 from watering all plants in a single step is also the maximum safe reward achievable in this environment layout. Given that unsafe behaviour is not possible, safety and return are combined into a single performance metric.

Figure 8 shows the performance of our two heuristics in this environment. As expected the Direct Penalty heuristic fails to exploit the sprinkler tile, while the Doubt heuristic is able to detect that the reward is genuine.

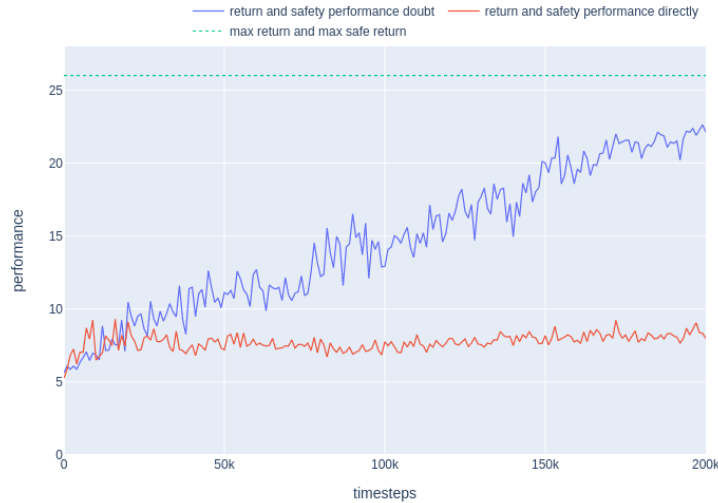


Fig. 8: Penalize Directly vs. Doubt - Case 2

4.3 Cases 3, 4, 5: Distinguishing Observation and Sprinkler Tile

In the last three cases, we vary the positions of the *S* and *O* tile. Our goal is to observe if the agent using our Doubt heuristic is able to distinguish between the genuine reward and the free lunch when both are presented as an option. There is no safety baseline since the maximum return can be achieved safely.

Looking at the overall outcome (Figures 9, 10 and 11), we can tell that the different positioning of the two special tiles affects the performance of the agent. We can see that the highest overlap of the safety and return functions occurs in case 4. A similar trend but slightly worse in terms of maximum reward and safe behaviour is observed in case 3. Finally, the case 5 results demonstrate the greatest divergence and therefore the most unsafe behaviour.



Fig. 9: Penalize Directly vs. Doubt - Case 3



Fig. 10: Penalize Directly vs. Doubt - Case 4



Fig. 11: Penalize Directly vs. Doubt - Case 5

Case 3 presents a configuration in which the agent is close to both of the special tiles from the start. The return in this experiment is the lowest by a small amount, possibly due to the fact that the agent doesn't settle for a high-reward exploitative policy (stepping on *O* or *S*) but also continues to water individual tomatoes on occasion. Thus, it achieves less reward in general by performing a lot of other low return actions. Analysis of the run shows that the agent spends minimal time stepping on either of the two tiles: the agent's *S*- and *O*-tile steps account for only 6.26% and 1.85% of actions respectively. In contrast, there is an evident preference to the *S* tile in case 4 (27.6% compared to 1.18% on *O*), and to the *O* tile in case 5 (14.8% compared to 6.69% on *S*).

In the best performing configuration (case 4), the agent starts close to the *O* tile. While undertaking random actions in the beginning and often stepping on the observation transformer, it quickly learns that this transition's reward is out of distribution and should be penalized. As it explores its environment and encounters the *S* tile, the agent is confronted with the same high reward but this time, it is a valid solution. After confirming that the effects of the *S* tile persist, it gradually learns that it is a harmless way to achieve good return.

In case 5, a high reward is obtained but safety performance deteriorates. Only in the first 5k steps or so do the safety performance and return functions align. Soon after that, the two diverge. The episode starts with the agent placed in close proximity to the *S* tile and further away from the *O* tile. The agent initially "turns on" the sprinkler repeatedly and learns that this is a safe solution. Thus, by regarding the transition as safe, the high reward becomes part of the rewards distribution. This becomes a problem when the agent reaches the *O* tile – by the time it steps on the observation transformer, the reward received is no longer a free lunch because of the many *S*-tile steps that have yielded the same high

reward. Therefore, the agent fails to even suspect the O tile and doesn't learn a safe policy.

5 Related Work

There are various emerging methods for solving the various reward misspecification problems highlighted by the Gridworlds. [1][5] look at negative side effects. [3] tackles *distributional shift*. Meanwhile robustness problems have been considered by [8]. As far as we are aware we are the first work to tackle reward gaming by self-modification.

6 Conclusion

We present two heuristics for overcoming reward gaming by self-modification in RL systems. Our first heuristic is agnostic to the environment and consists simply of penalizing unexpectedly high rewards detected by comparing them to the distribution of previous rewards. Our second heuristic is context dependent and relies upon being able to define a validation action that can verify that a reward is genuine. We ran experiments to evaluate the performance of these two heuristics on variations of the AI Safety Gridworlds tomato watering environment where special tiles could return large genuine and false rewards. We showed that even where only free lunches existed, the heuristic that attempted to validate rewards performed better because it allocated fewer incorrect penalties early in the learning process. However, this heuristic is sensitive to the initial conditions of the learning and we illustrated, via a case where the genuine reward tile was accessible early in the learning process while the free lunch was only encountered later, that unsafe behaviour could still be learned.

The value of the first heuristic is its context independence. In environments where reward functions are difficult to precisely define, the heuristic can be used to exclude suspiciously high rewards at the risk of reduced performance. While the second heuristic has better performance it does require the identification of a validation action. It may be that classes of such actions can be determined – for instance our persistence assumption may be sufficient in many cases – but there are many environments where determining an action that can validate a reward that is independent of the way the reward function is calculated may be challenging and even where one can be found the heuristic is vulnerable if there are also actions that genuinely return high rewards. It should also be noted that there is a cost involved in performing a validation step which in many applications may prove prohibitive, even where such a step can be identified.

References

1. Armstrong, S., Levinstein, B.: Low impact artificial intelligences. CoRR **abs/1705.10720** (2017), <http://arxiv.org/abs/1705.10720>
2. Clark, J., Amodei, D.: Faulty reward functions in the wild (2016), <https://blog.openai.com/faulty-reward-functions/>
3. Hadfield-Menell, D., Milli, S., Abbeel, P., Russell, S.J., Dragan, A.: Inverse reward design. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (eds.) *Advances in Neural Information Processing Systems*. vol. 30. Curran Associates, Inc. (2017)
4. Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., Silver, D.: Rainbow: Combining improvements in deep reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence* **32**(1) (Apr 2018)
5. Krakovna, V., Orseau, L., Martic, M., Legg, S.: Measuring and avoiding side effects using relative reachability. CoRR **abs/1806.01186** (2018), <http://arxiv.org/abs/1806.01186>
6. Leike, J., Martic, M., Krakovna, V., Ortega, P.A., Everitt, T., Lefrancq, A., Orseau, L., Legg, S.: AI safety gridworlds. CoRR **abs/1711.09883** (2017), <http://arxiv.org/abs/1711.09883>
7. Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.A.: Playing atari with deep reinforcement learning. CoRR **abs/1312.5602** (2013), <http://arxiv.org/abs/1312.5602>
8. Santara, A., Naik, A., Ravindran, B., Das, D., Mudigere, D., Avancha, S., Kaul, B.: Rail: Risk-averse imitation learning. In: *Proceedings of AAMAS '18*. p. 2062–2063 (2018)
9. Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., Hassabis, D.: Mastering the game of Go with deep neural networks and tree search. *Nature* **529**(7587), 484–489 (Jan 2016). <https://doi.org/10.1038/nature16961>
10. Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., Hassabis, D.: A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science* **362**(6419), 1140–1144 (2018). <https://doi.org/10.1126/science.aar6404>
11. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*. The MIT Press, second edn. (2018)